📜 The 7 Principles of Testing

The Laws of QA

✍️ **Author:** Georgia Antoniou

🏷️ **Tags:** QA, ISTQB, Fundamentals

⏱️ **Reading Time:** 4 min

🚀 TL;DR

1. You can't prove software is bug-free.
2. You can't test *everything*.
3. Test early.
4. Bugs hide in clusters.
5. Old tests stop finding new bugs (Pesticide Paradox).
6. Testing depends on the context.
7. A bug-free app is useless if it's the wrong app.

🧐 The Core Principles Explained

Just like Physics has laws, Software Testing has these 7 absolute truths.

1. Testing shows the presence of defects, not their absence

Testing proves that bugs *exist*. It cannot prove that bugs *don't* exist. Even if I find 0 bugs, I cannot guarantee the software is perfect. I can only say "I didn't find any yet."

2. Exhaustive testing is impossible

You cannot test every single input (1, 2, 3... 1,000,000). It would take 100 years.

Solution: We use **Risk-Based Testing** to test the most important parts.

3. Early testing saves time and money

The earlier you find a bug, the cheaper it is to fix. (We covered this in the **Shift Left** article!).

4. Defect Clustering (The Pareto Principle)

80% of the bugs are found in 20% of the modules.

Bugs are social; they like to hang out together. If you find one bug in the "Search" feature, keep looking there - there are likely ten more.

5. The Pesticide Paradox

If you use the same pesticide on crops, the insects eventually become immune.

If you run the exact same manual tests every day, you will stop finding new bugs.

Solution: You must constantly update and change your test cases.

6. Testing is context dependent

Testing a **Mobile Game** is different from testing a **Heart Pacemaker**.

- Game: Focus on Fun and Graphics.
- Pacemaker: Focus on Safety and Reliability. No risk is allowed.

7. The "Absence-of-Errors" Fallacy

Finding and fixing all the bugs doesn't help if you built a product nobody wants.

- *Example:* We built a bug-free app for "VHS Tape Rentals." It works perfectly, but... nobody uses VHS anymore.

🌾 The Pesticide Paradox Analogy

Imagine you clean your house. You always dust the TV, vacuum the rug, and wipe the table. The house looks clean.

But you **never** look behind the sofa.

Over time, the dust bunnies (bugs) accumulate behind the sofa because your "Cleaning Test" never checks there.

To find new dirt, you have to look in new places.

💡 Real World Example: Defect Clustering

Scenario: An E-Commerce Website.

I start testing and find 3 bugs in the **"Coupon Code"** field.

- *My Instinct:* "Okay, I found the bugs there, I'll move to the 'Logout' button."
- *The Principle:* "No! Since there are 3 bugs in Coupon Code, it means that code is messy. There are probably 10 more."
- *Action:* I spend **more** time on the Coupon Code field and find a critical crash.

🧠 Why This Matters

Recruiters often ask: *"Why didn't you test every possible combination?"*

If you don't know the principles, you might apologize.

If you **do** know them, you say: *"Because Principle #2 says Exhaustive Testing is impossible. Instead, I prioritized the critical risks."*